

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

*I D.NAVADEEP/192371068 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

D.NAVADEEP

Annexure A

Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
- B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).
- C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation

A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- A. Identify issues in prime generation, key generation, and modular exponentiation.
- B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- A. Debug incorrect shared key generation due to improper modular arithmetic.
- B. Optimize exponentiation operations for large prime values.
- C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

- A. Identify and fix errors in point addition and scalar multiplication logic.
- B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

- C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

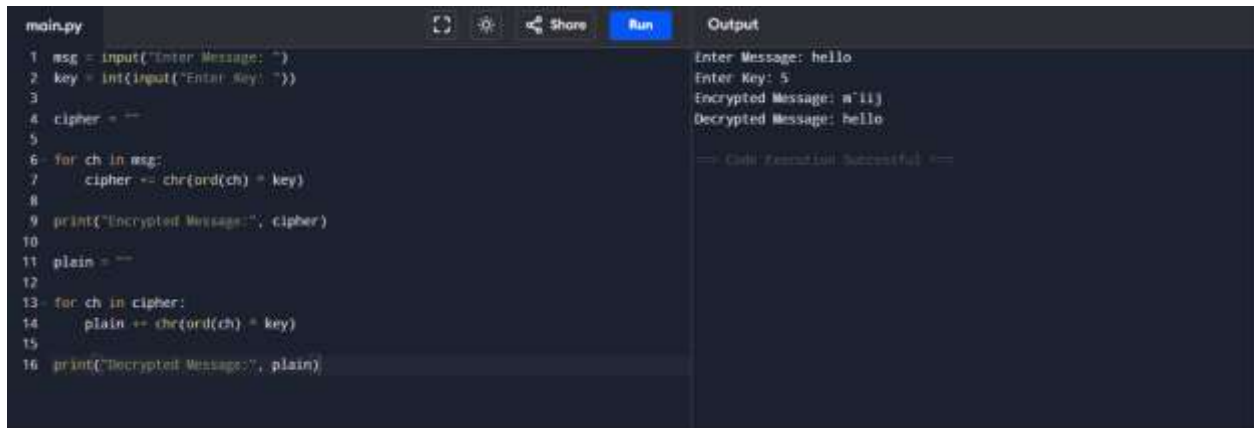
RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- D. Identify and fix logical and implementation errors in key scheduling and permutation steps.
- E. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).
- F. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.



```
main.py
1 msg = input("Enter Message: ")
2 key = int(input("Enter Key: "))
3
4 cipher = ""
5
6 for ch in msg:
7     cipher += chr(ord(ch) ^ key)
8
9 print("Encrypted Message:", cipher)
10
11 plain = ""
12
13 for ch in cipher:
14     plain += chr(ord(ch) ^ key)
15
16 print("Decrypted Message:", plain)
```

Output

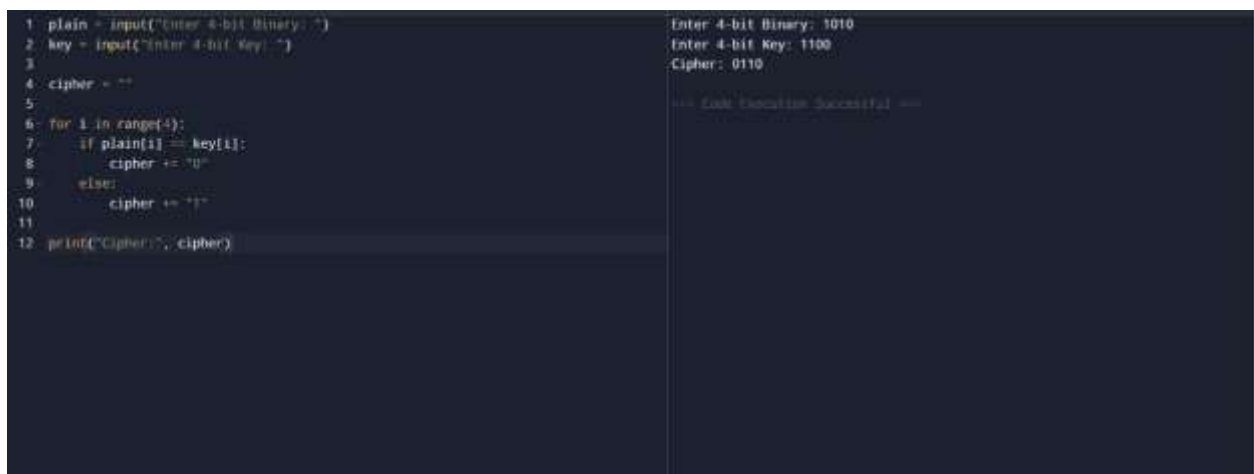
```
Enter Message: hello
Enter Key: 5
Encrypted Message: n`ii`
Decrypted Message: hello

--- Code Execution Successful ---
```

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- a. Debug incorrect padding and IV handling causing decryption failures.
- b. Refactor the code to minimize redundant computations and memory usage.
- c. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.



```
1 plain = input("Enter 4-bit Binary: ")
2 key = input("Enter 4-bit Key: ")
3
4 cipher = ""
5
6 for i in range(4):
7     if plain[i] == key[i]:
8         cipher += "0"
9     else:
10        cipher += "1"
11
12 print("Cipher:", cipher)
```

Output

```
Enter 4-bit Binary: 1010
Enter 4-bit Key: 1100
Cipher: 0110

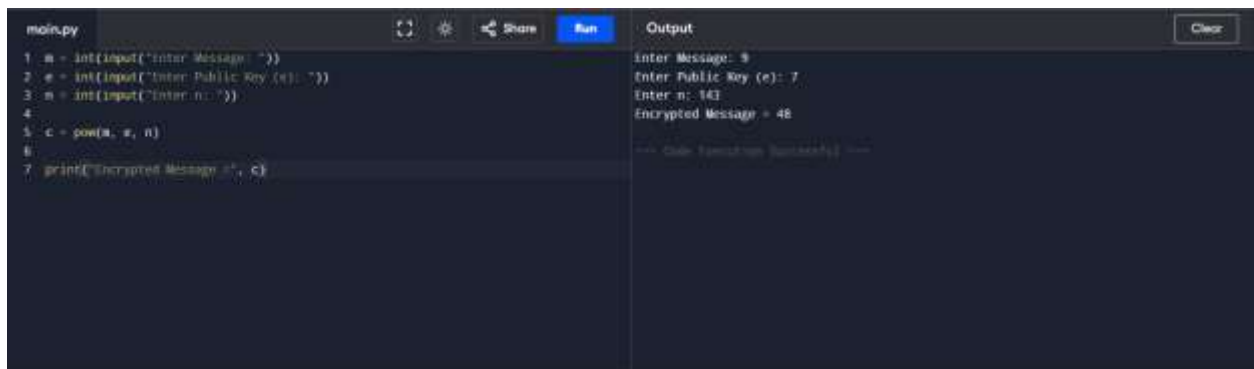
--- Code Execution Successful ---
```

3. Debugging RSA Cryptosystem Implementation

A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- a. Identify issues in prime generation, key generation, and modular exponentiation.
- b. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- c. Evaluate the trade-offs between key size, security, and computational

performance.



```
main.py
1 a = int(input("Enter Message: "))
2 e = int(input("Enter Public Key (e): "))
3 n = int(input("Enter n: "))
4
5 c = pow(a, e, n)
6
7 print("Encrypted Message: ", c)
```

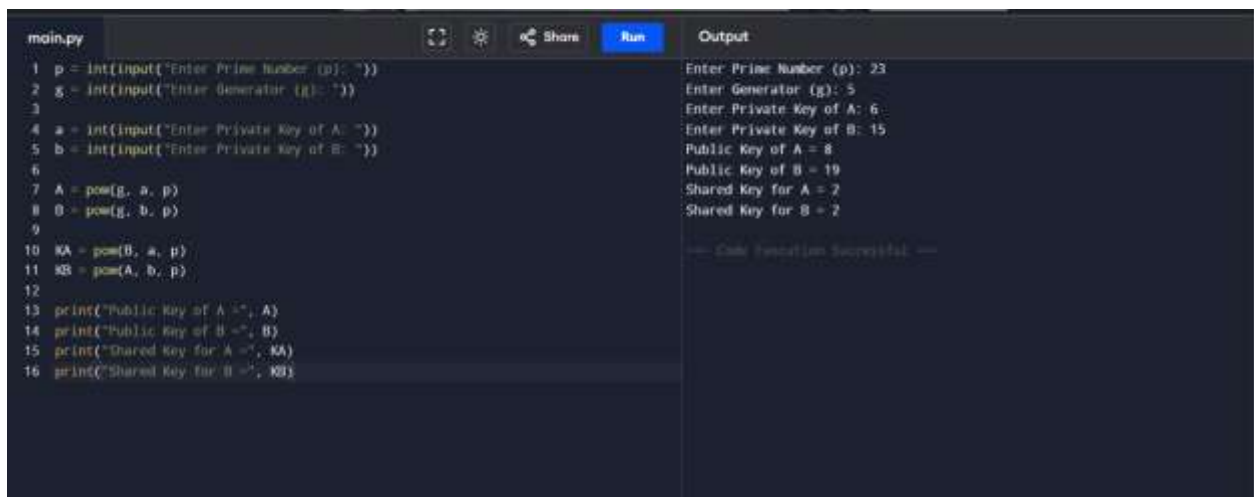
```
Output
Enter Message: 4
Enter Public Key (e): 7
Enter n: 143
Encrypted Message: 48

--- Code Execution Successful ---
```

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A

Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- Debug incorrect shared key generation due to improper modular arithmetic.
- Optimize exponentiation operations for large prime values.
- Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.



```
main.py
1 p = int(input("Enter Prime Number (p): "))
2 g = int(input("Enter Generator (g): "))
3
4 a = int(input("Enter Private Key of A: "))
5 b = int(input("Enter Private Key of B: "))
6
7 A = pow(g, a, p)
8 B = pow(g, b, p)
9
10 KA = pow(B, a, p)
11 KB = pow(A, b, p)
12
13 print("Public Key of A: ", A)
14 print("Public Key of B: ", B)
15 print("Shared Key for A: ", KA)
16 print("Shared Key for B: ", KB)
```

```
Output
Enter Prime Number (p): 23
Enter Generator (g): 5
Enter Private Key of A: 6
Enter Private Key of B: 15
Public Key of A: 8
Public Key of B: 19
Shared Key for A: 2
Shared Key for B: 2

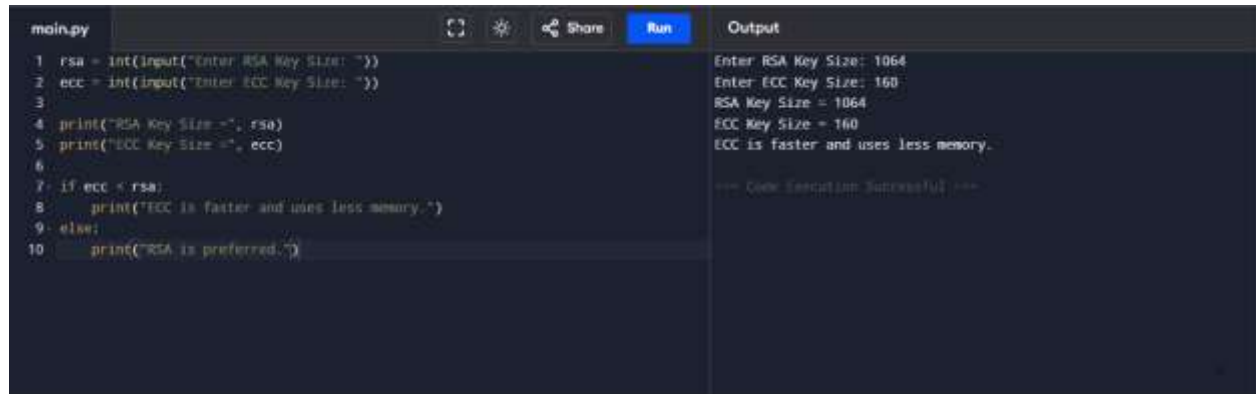
--- Code Execution Successful ---
```

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A

Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

- Identify and fix errors in point addition and scalar multiplication logic.
- Optimize the implementation using efficient algorithms (e.g., double-and-add).

- c. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.



```
main.py  [Icons]  Share  Run  Output
1  rsa = int(input("Enter RSA Key Size: "))
2  ecc = int(input("Enter ECC Key Size: "))
3
4  print("RSA Key Size =", rsa)
5  print("ECC Key Size =", ecc)
6
7  if ecc < rsa:
8      print("ECC is faster and uses less memory.")
9  else:
10     print("RSA is preferred.")
```

Enter RSA Key Size: 1064
Enter ECC Key Size: 160
RSA Key Size = 1064
ECC Key Size = 160
ECC is faster and uses less memory.
=== Code Execution Successful ===